

```

1  class TreeNode {
2      private int data;
3      public TreeNode left;
4      public TreeNode right;
5      private int size = 0;
6
7      public TreeNode(int d) {
8          data = d;
9          size = 1;
10     }
11
12     public TreeNode getRandomNode() {
13         int leftSize = left == null ? 0 : left.size();
14         Random random = new Random();
15         int index = random.nextInt(size);
16         if (index < leftSize) {
17             return left.getRandomNode();
18         } else if (index == leftSize) {
19             return this;
20         } else {
21             return right.getRandomNode();
22         }
23     }
24
25     public void insertInOrder(int d) {
26         if (d <= data) {
27             if (left == null) {
28                 left = new TreeNode(d);
29             } else {
30                 left.insertInOrder(d);
31             }
32         } else {
33             if (right == null) {
34                 right = new TreeNode(d);
35             } else {
36                 right.insertInOrder(d);
37             }
38         }
39         size++;
40     }
41
42     public int size() { return size; }
43     public int data() { return data; }
44
45     public TreeNode find(int d) {
46         if (d == data) {
47             return this;
48         } else if (d <= data) {
49             return left != null ? left.find(d) : null;
50         } else if (d > data) {
51             return right != null ? right.find(d) : null;
52         }
53         return null;
54     }
55 }

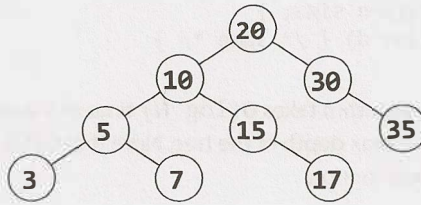
```

In a balanced tree, this algorithm will be $O(\log N)$, where N is the number of nodes.

Option #7 [Fast & Working]

Random number calls can be expensive. If we'd like, we can reduce the number of random number calls substantially.

Imagine we called `getRandomNode` on the tree below, and then traversed left.



We traversed left because we picked a number between 0 and 5 (inclusive). When we traverse left, we again pick a random number between 0 and 5. Why re-pick? The first number will work just fine.

But what if we went right instead? We have a number between 7 and 8 (inclusive) but we would need a number between 0 and 1 (inclusive). That's easy to fix: just subtract out `LEFT_SIZE + 1`.

Another way to think about what we're doing is that the initial random number call indicates which node (*i*) to return, and then we're locating the *i*th node in an in-order traversal. Subtracting `LEFT_SIZE + 1` from *i* reflects that, when we go right, we skip over `LEFT_SIZE + 1` nodes in the in-order traversal.

```

1  class Tree {
2      TreeNode root = null;
3
4      public int size() { return root == null ? 0 : root.size(); }
5
6      public TreeNode getRandomNode() {
7          if (root == null) return null;
8
9          Random random = new Random();
10         int i = random.nextInt(size());
11         return root.getIthNode(i);
12     }
13
14     public void insertInOrder(int value) {
15         if (root == null) {
16             root = new TreeNode(value);
17         } else {
18             root.insertInOrder(value);
19         }
20     }
21 }
22
23 class TreeNode {
24     /* constructor and variables are the same. */
25
26     public TreeNode getIthNode(int i) {
27         int leftSize = left == null ? 0 : left.size();
28         if (i < leftSize) {
29             return left.getIthNode(i);
30         } else if (i == leftSize) {
31             return this;
32         } else {

```

```

33         /* Skipping over leftSize + 1 nodes, so subtract them. */
34         return right.getIthNode(i - (leftSize + 1));
35     }
36 }
37
38 public void insertInOrder(int d) { /* same */ }
39 public int size() { return size; }
40 public TreeNode find(int d) { /* same */ }
41 }

```

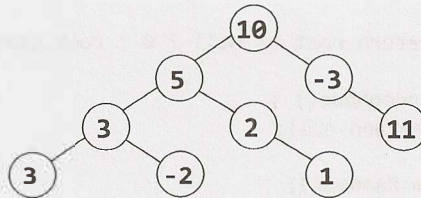
Like the previous algorithm, this algorithm takes $O(\log N)$ time in a balanced tree. We can also describe the runtime as $O(D)$, where D is the max depth of the tree. Note that $O(D)$ is an accurate description of the runtime whether the tree is balanced or not.

4.12 Paths with Sum: You are given a binary tree in which each node contains an integer value (which might be positive or negative). Design an algorithm to count the number of paths that sum to a given value. The path does not need to start or end at the root or a leaf, but it must go downwards (traveling only from parent nodes to child nodes).

pg 111

SOLUTION

Let's pick a potential sum—say, 8—and then draw a binary tree based on this. This tree intentionally has a number of paths with this sum.



One option is the brute force approach.

Solution #1: Brute Force

In the brute force approach, we just look at all possible paths. To do this, we traverse to each node. At each node, we recursively try all paths downwards, tracking the sum as we go. As soon as we hit our target sum, we increment the total.

```

1  int countPathsWithSum(TreeNode root, int targetSum) {
2      if (root == null) return 0;
3
4      /* Count paths with sum starting from the root. */
5      int pathsFromRoot = countPathsWithSumFromNode(root, targetSum, 0);
6
7      /* Try the nodes on the left and right. */
8      int pathsOnLeft = countPathsWithSum(root.left, targetSum);
9      int pathsOnRight = countPathsWithSum(root.right, targetSum);
10
11     return pathsFromRoot + pathsOnLeft + pathsOnRight;
12 }
13
14 /* Returns the number of paths with this sum starting from this node. */

```

```

15 int countPathsWithSumFromNode(TreeNode node, int targetSum, int currentSum) {
16     if (node == null) return 0;
17
18     currentSum += node.data;
19
20     int totalPaths = 0;
21     if (currentSum == targetSum) { // Found a path from the root
22         totalPaths++;
23     }
24
25     totalPaths += countPathsWithSumFromNode(node.left, targetSum, currentSum);
26     totalPaths += countPathsWithSumFromNode(node.right, targetSum, currentSum);
27     return totalPaths;
28 }

```

What is the time complexity of this algorithm?

Consider that node at depth d will be “touched” (via `countPathsWithSumFromNode`) by d nodes above it.

In a balanced binary tree, d will be no more than approximately $\log N$. Therefore, we know that with N nodes in the tree, `countPathsWithSumFromNode` will be called $O(N \log N)$ times. The runtime is $O(N \log N)$.

We can also approach this from the other direction. At the root node, we traverse to all $N - 1$ nodes beneath it (via `countPathsWithSumFromNode`). At the second level (where there are two nodes), we traverse to $N - 3$ nodes. At the third level (where there are four nodes, plus three above those), we traverse to $N - 7$ nodes. Following this pattern, the total work is roughly:

$$(N - 1) + (N - 3) + (N - 7) + (N - 15) + (N - 31) + \dots + (N - N)$$

To simplify this, notice that the left side of each term is always N and the right side is one less than a power of two. The number of terms is the depth of the tree, which is $O(\log N)$. For the right side, we can ignore the fact that it's one less than a power of two. Therefore, we really have this:

$$\begin{aligned}
 &O(N * [\text{number of terms}] - [\text{sum of powers of two from 1 through } N]) \\
 &O(N \log N - N) \\
 &O(N \log N)
 \end{aligned}$$

If the value of the sum of powers of two from 1 through N isn't obvious to you, think about what the powers of two look like in binary:

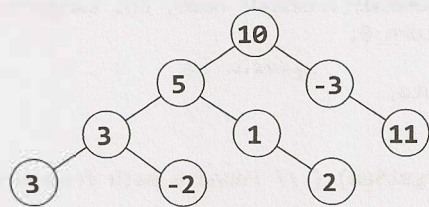
$$\begin{array}{r}
 0001 \\
 + 0010 \\
 + 0100 \\
 + 1000 \\
 \hline
 = 1111
 \end{array}$$

Therefore, the runtime is $O(N \log N)$ in a balanced tree.

In an unbalanced tree, the runtime could be much worse. Consider a tree that is just a straight line down. At the root, we traverse to $N - 1$ nodes. At the next level (with just a single node), we traverse to $N - 2$ nodes. At the third level, we traverse to $N - 3$ nodes, and so on. This leads us to the sum of numbers between 1 and N , which is $O(N^2)$.

Solution #2: Optimized

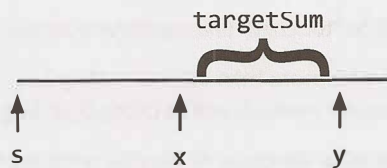
In analyzing the last solution, we may realize that we repeat some work. For a path such as $10 \rightarrow 5 \rightarrow 3 \rightarrow -2$, we traverse this path (or parts of it) repeatedly. We do it when we start with node 10, then when we go to node 5 (looking at 5, then 3, then -2), then when we go to node 3, and then finally when we go to node -2. Ideally, we'd like to reuse this work.



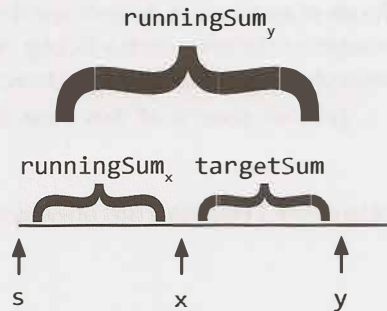
Let’s isolate a given path and treat it as just an array. Consider a (hypothetical, extended) path like:

10 -> 5 -> 1 -> 2 -> -1 -> -1 -> 7 -> 1 -> 2

What we’re really saying then is: How many contiguous subsequences in this array sum to a target sum such as 8? In other words, for each *y*, we’re trying to find the *x* values below. (Or, more accurately, the number of *x* values below.)



If each value knows its running sum (the sum of values from *s* through itself), then we can find this pretty easily. We just need to leverage this simple equation: $runningSum_x = runningSum_y - targetSum$. We then look for the values of *x* where this is true.



Since we’re just looking for the number of paths, we can use a hash table. As we iterate through the array, build a hash table that maps from a *runningSum* to the number of times we’ve seen that sum. Then, for each *y*, look up $runningSum_y - targetSum$ in the hash table. The value in the hash table will tell you the number of paths with sum *targetSum* that end at *y*.

For example:

index:	0	1	2	3	4	5	6	7	8
value:	10	-> 5	-> 1	-> 2	-> -1	-> -1	-> 7	-> 1	-> 2
sum:	10	15	16	18	17	16	23	24	26

The value of *runningSum_y* is 24. If *targetSum* is 8, then we’d look up 16 in the hash table. This would have a value of 2 (originating from index 2 and index 5). As we can see above, indexes 3 through 7 and indexes 6 through 7 have sums of 8.

Now that we’ve settled the algorithm for an array, let’s review this on a tree. We take a similar approach.

We traverse through the tree using depth-first search. As we visit each node:

1. Track its `runningSum`. We'll take this in as a parameter and immediately increment it by `node.value`.
2. Look up `runningSum - targetSum` in the hash table. The value there indicates the total number. Set `totalPaths` to this value.
3. If `runningSum == targetSum`, then there's one additional path that starts at the root. Increment `totalPaths`.
4. Add `runningSum` to the hash table (incrementing the value if it's already there).
5. Recurse left and right, counting the number of paths with sum `targetSum`.
6. After we're done recursing left and right, decrement the value of `runningSum` in the hash table. This is essentially backing out of our work; it reverses the changes to the hash table so that other nodes don't use it (since we're now done with `node`).

Despite the complexity of deriving this algorithm, the code to implement this is relatively simple.

```

1  int countPathsWithSum(TreeNode root, int targetSum) {
2      return countPathsWithSum(root, targetSum, 0, new HashMap<Integer, Integer>());
3  }
4
5  int countPathsWithSum(TreeNode node, int targetSum, int runningSum,
6                      HashMap<Integer, Integer> pathCount) {
7      if (node == null) return 0; // Base case
8
9      /* Count paths with sum ending at the current node. */
10     runningSum += node.data;
11     int sum = runningSum - targetSum;
12     int totalPaths = pathCount.getOrDefault(sum, 0);
13
14     /* If runningSum equals targetSum, then one additional path starts at root.
15      * Add in this path.*/
16     if (runningSum == targetSum) {
17         totalPaths++;
18     }
19
20     /* Increment pathCount, recurse, then decrement pathCount. */
21     incrementHashTable(pathCount, runningSum, 1); // Increment pathCount
22     totalPaths += countPathsWithSum(node.left, targetSum, runningSum, pathCount);
23     totalPaths += countPathsWithSum(node.right, targetSum, runningSum, pathCount);
24     incrementHashTable(pathCount, runningSum, -1); // Decrement pathCount
25
26     return totalPaths;
27 }
28
29 void incrementHashTable(HashMap<Integer, Integer> hashTable, int key, int delta) {
30     int newCount = hashTable.getOrDefault(key, 0) + delta;
31     if (newCount == 0) { // Remove when zero to reduce space usage
32         hashTable.remove(key);
33     } else {
34         hashTable.put(key, newCount);
35     }
36 }

```

The runtime for this algorithm is $O(N)$, where N is the number of nodes in the tree. We know it is $O(N)$ because we travel to each node just once, doing $O(1)$ work each time. In a balanced tree, the space complexity is $O(\log N)$ due to the hash table. The space complexity can grow to $O(n)$ in an unbalanced tree.

5

Solutions to Bit Manipulation

5.1 Insertion: You are given two 32-bit numbers, *N* and *M*, and two bit positions, *i* and *j*. Write a method to insert *M* into *N* such that *M* starts at bit *j* and ends at bit *i*. You can assume that the bits *j* through *i* have enough space to fit all of *M*. That is, if *M* = 10011, you can assume that there are at least 5 bits between *j* and *i*. You would not, for example, have *j* = 3 and *i* = 2, because *M* could not fully fit between bit 3 and bit 2.

EXAMPLE

Input: *N* = 10000000000, *M* = 10011, *i* = 2, *j* = 6

Output: *N* = 10001001100

pg 115

SOLUTION

This problem can be approached in three key steps:

1. Clear the bits *j* through *i* in *N*
2. Shift *M* so that it lines up with bits *j* through *i*
3. Merge *M* and *N*.

The trickiest part is Step 1. How do we clear the bits in *N*? We can do this with a mask. This mask will have all 1s, except for 0s in the bits *j* through *i*. We create this mask by creating the left half of the mask first, and then the right half.

```
1  int updateBits(int n, int m, int i, int j) {
2      /* Create a mask to clear bits i through j in n. EXAMPLE: i = 2, j = 4. Result
3       * should be 11100011. For simplicity, we'll use just 8 bits for the example. */
4      int allOnes = ~0; // will equal sequence of all 1s
5
6      // 1s before position j, then 0s. left = 11100000
7      int left = allOnes << (j + 1);
8
9      // 1's after position i. right = 00000011
10     int right = ((1 << i) - 1);
11
12     // All 1s, except for 0s between i and j. mask = 11100011
13     int mask = left | right;
14
15     /* Clear bits j through i then put m in there */
16     int n_cleared = n & mask; // Clear bits j through i.
17     int m_shifted = m << i; // Move m into correct position.
```

```

18
19     return n_cleared | m_shifted; // OR them, and we're done!
20 }

```

In a problem like this (and many bit manipulation problems), you should make sure to thoroughly test your code. It's extremely easy to wind up with off-by-one errors.

5.2 Binary to String: Given a real number between 0 and 1 (e.g., 0.72) that is passed in as a double, print the binary representation. If the number cannot be represented accurately in binary with at most 32 characters, print "ERROR."

pg 116

SOLUTION

NOTE: When otherwise ambiguous, we'll use the subscripts x_2 and x_{10} to indicate whether x is in base 2 or base 10.

First, let's start off by asking ourselves what a non-integer number in binary looks like. By analogy to a decimal number, the binary number 0.101_2 would look like:

$$0.101_2 = 1 * \frac{1}{2^1} + 0 * \frac{1}{2^2} + 1 * \frac{1}{2^3}.$$

To print the decimal part, we can multiply by 2 and check if $2n$ is greater than or equal to 1. This is essentially "shifting" the fractional sum. That is:

$$\begin{aligned} r &= 2_{10} * n \\ &= 2_{10} * 0.101_2 \\ &= 1 * \frac{1}{2^0} + 0 * \frac{1}{2^1} + 1 * \frac{1}{2^2} \\ &= 1.01_2 \end{aligned}$$

If $r \geq 1$, then we know that n had a 1 right after the decimal point. By doing this continuously, we can check every digit.

```

1  String printBinary(double num) {
2      if (num >= 1 || num <= 0) {
3          return "ERROR";
4      }
5
6      StringBuilder binary = new StringBuilder();
7      binary.append(".");
8      while (num > 0) {
9          /* Setting a limit on length: 32 characters */
10         if (binary.length() >= 32) {
11             return "ERROR";
12         }
13
14         double r = num * 2;
15         if (r >= 1) {
16             binary.append(1);
17             num = r - 1;
18         } else {
19             binary.append(0);
20             num = r;
21         }
22     }
23     return binary.toString();
24 }

```


Alternatively, rather than multiplying the number by two and comparing it to 1, we can compare the number to .5, then .25, and so on. The code below demonstrates this approach.

```

1  String printBinary2(double num) {
2      if (num >= 1 || num <= 0) {
3          return "ERROR";
4      }
5
6      StringBuilder binary = new StringBuilder();
7      double frac = 0.5;
8      binary.append(".");
9      while (num > 0) {
10         /* Setting a limit on length: 32 characters */
11         if (binary.length() > 32) {
12             return "ERROR";
13         }
14         if (num >= frac) {
15             binary.append(1);
16             num -= frac;
17         } else {
18             binary.append(0);
19         }
20         frac /= 2;
21     }
22     return binary.toString();
23 }
```

Both approaches are equally good; choose the one you feel most comfortable with.

Either way, you should make sure to prepare thorough test cases for this problem—and to actually run through them in your interview.

5.3 Flip Bit to Win: You have an integer and you can flip exactly one bit from a 0 to a 1. Write code to find the length of the longest sequence of 1s you could create.

EXAMPLE

Input: 1775 (or: 11011101111)

Output: 8

pg 116

SOLUTION

We can think about each integer as being an alternating sequence of 0s and 1s. Whenever a 0s sequence has length one, we can potentially merge the adjacent 1s sequences.

Brute Force

One approach is to convert an integer into an array that reflects the lengths of the 0s and 1s sequences. For example, 11011101111 would be (reading from right to left) $[0_0, 4_1, 1_0, 3_1, 1_0, 2_1, 21_0]$. The subscript reflects whether the integer corresponds to a 0s sequence or a 1s sequence, but the actual solution doesn't need this. It's a strictly alternating sequence, always starting with the 0s sequence.

Once we have this, we just walk through the array. At each 0s sequence, then we consider merging the adjacent 1s sequences if the 0s sequence has length 1.

```

1  int longestSequence(int n) {
```

```

2     if (n == -1) return Integer.BYTES * 8;
3     ArrayList<Integer> sequences = getAlternatingSequences(n);
4     return findLongestSequence(sequences);
5 }
6
7  /* Return a list of the sizes of the sequences. The sequence starts off with the
8     number of 0s (which might be 0) and then alternates with the counts of each
9     value.*/
10 ArrayList<Integer> getAlternatingSequences(int n) {
11     ArrayList<Integer> sequences = new ArrayList<Integer>();
12
13     int searchingFor = 0;
14     int counter = 0;
15
16     for (int i = 0; i < Integer.BYTES * 8; i++) {
17         if ((n & 1) != searchingFor) {
18             sequences.add(counter);
19             searchingFor = n & 1; // Flip 1 to 0 or 0 to 1
20             counter = 0;
21         }
22         counter++;
23         n >>= 1;
24     }
25     sequences.add(counter);
26
27     return sequences;
28 }
29
30 /* Given the lengths of alternating sequences of 0s and 1s, find the longest one
31    * we can build. */
32 int findLongestSequence(ArrayList<Integer> seq) {
33     int maxSeq = 1;
34
35     for (int i = 0; i < seq.size(); i += 2) {
36         int zerosSeq = seq.get(i);
37         int onesSeqRight = i - 1 >= 0 ? seq.get(i - 1) : 0;
38         int onesSeqLeft = i + 1 < seq.size() ? seq.get(i + 1) : 0;
39
40         int thisSeq = 0;
41         if (zerosSeq == 1) { // Can merge
42             thisSeq = onesSeqLeft + 1 + onesSeqRight;
43         } if (zerosSeq > 1) { // Just add a zero to either side
44             thisSeq = 1 + Math.max(onesSeqRight, onesSeqLeft);
45         } else if (zerosSeq == 0) { // No zero, but take either side
46             thisSeq = Math.max(onesSeqRight, onesSeqLeft);
47         }
48         maxSeq = Math.max(thisSeq, maxSeq);
49     }
50
51     return maxSeq;
52 }

```

This is pretty good. It's $O(b)$ time and $O(b)$ memory, where b is the length of the sequence.

Be careful with how you express the runtime. For example, if you say the runtime is $O(n)$, what is n ? It is not correct to say that this algorithm is $O(\text{value of the integer})$. This algorithm is $O(\text{number of bits})$. For this reason, when you have potential ambiguity in what n might mean, it's best just to not use n . Then neither you nor your interviewer will be confused. Pick a different variable name. We used "b", for the number of bits. Something logical works well.

Can we do better? Recall the concept of Best Conceivable Runtime. The B.C.R. for this algorithm is $O(b)$ (since we'll always have to read through the sequence), so we know we can't optimize the time. We can, however, reduce the memory usage.

Optimal Algorithm

To reduce the space usage, note that we don't need to hang on to the length of each sequence the entire time. We only need it long enough to compare each 1s sequence to the immediately preceding 1s sequence.

Therefore, we can just walk through the integer doing this, tracking the current 1s sequence length and the previous 1s sequence length. When we see a zero, update `previousLength`:

- If the next bit is a 1, `previousLength` should be set to `currentLength`.
- If the next bit is a 0, then we can't merge these sequences together. So, set `previousLength` to 0.

Update `maxLength` as we go.

```

1  int flipBit(int a) {
2      /* If all 1s, this is already the longest sequence. */
3      if (~a == 0) return Integer.BYTES * 8;
4
5      int currentLength = 0;
6      int previousLength = 0;
7      int maxLength = 1; // We can always have a sequence of at least one 1
8      while (a != 0) {
9          if ((a & 1) == 1) { // Current bit is a 1
10             currentLength++;
11         } else if ((a & 1) == 0) { // Current bit is a 0
12             /* Update to 0 (if next bit is 0) or currentLength (if next bit is 1). */
13             previousLength = (a & 2) == 0 ? 0 : currentLength;
14             currentLength = 0;
15         }
16         maxLength = Math.max(previousLength + currentLength + 1, maxLength);
17         a >>= 1;
18     }
19     return maxLength;
20 }
```

The runtime of this algorithm is still $O(b)$, but we use only $O(1)$ additional memory.

5.4 Next Number: Given a positive integer, print the next smallest and the next largest number that have the same number of 1 bits in their binary representation.

pg 116

SOLUTION

There are a number of ways to approach this problem, including using brute force, using bit manipulation, and using clever arithmetic. Note that the arithmetic approach builds on the bit manipulation approach. You'll want to understand the bit manipulation approach before going on to the arithmetic one.

The terminology can be confusing for this problem. We'll call `getNext` the bigger number and `getPrev` the smaller number.

The Brute Force Approach

An easy approach is simply brute force: count the number of 1s in n , and then increment (or decrement) until you find a number with the same number of 1s. Easy—but not terribly interesting. Can we do something a bit more optimal? Yes!

Let's start with the code for `getNext`, and then move on to `getPrev`.

Bit Manipulation Approach for Get Next Number

If we think about what the next number *should* be, we can observe the following. Given the number 13948, the binary representation looks like:

1	1	0	1	1	0	0	1	1	1	1	1	0	0
13	12	11	10	9	8	7	6	5	4	3	2	1	0

We want to make this number bigger (but not *too* big). We also need to keep the same number of ones.

Observation: Given a number n and two bit locations i and j , suppose we flip bit i from a 1 to a 0, and bit j from a 0 to a 1. If $i > j$, then n will have decreased. If $i < j$, then n will have increased.

We know the following:

1. If we flip a zero to a one, we must flip a one to a zero.
2. When we do that, the number will be bigger if and only if the zero-to-one bit was to the left of the one-to-zero bit.
3. We want to make the number bigger, but not unnecessarily bigger. Therefore, we need to flip the rightmost zero which has ones on the right of it.

To put this in a different way, we are flipping the rightmost non-trailing zero. That is, using the above example, the trailing zeros are in the 0th and 1st spot. The rightmost non-trailing zero is at bit 7. Let's call this position p .

Step 1: Flip rightmost non-trailing zero

1	1	0	1	1	0	1	1	1	1	1	1	0	0
13	12	11	10	9	8	7	6	5	4	3	2	1	0

With this change, we have increased the size of n . But, we also have one too many ones, and one too few zeros. We'll need to shrink the size of our number as much as possible while keeping that in mind.

We can shrink the number by rearranging all the bits to the right of bit p such that the 0s are on the left and the 1s are on the right. As we do this, we want to replace one of the 1s with a 0.

A relatively easy way of doing this is to count how many ones are to the right of p , clear all the bits from 0 until p , and then add back in $c1-1$ ones. Let $c1$ be the number of ones to the right of p and $c0$ be the number of zeros to the right of p .

Let's walk through this with an example.

Step 2: Clear bits to the right of p . From before, $c0 = 2$, $c1 = 5$, $p = 7$.

1	1	0	1	1	0	1	0	0	0	0	0	0	0
13	12	11	10	9	8	7	6	5	4	3	2	1	0

To clear these bits, we need to create a mask that is a sequence of ones, followed by p zeros. We can do this as follows:

```
a = 1 << p; // all zeros except for a 1 at position p.
b = a - 1;  // all zeros, followed by p ones.
mask = ~b;  // all ones, followed by p zeros.
n = n & mask; // clears rightmost p bits.
```

Or, more concisely, we do:

```
n &= ~(1 << p) - 1;
```

Step 3: Add in $c1 - 1$ ones.

1	1	0	1	1	0	1	0	0	0	1	1	1	1
13	12	11	10	9	8	7	6	5	4	3	2	1	0

To insert $c1 - 1$ ones on the right, we do the following:

```
a = 1 << (c1 - 1); // 0s with a 1 at position c1 - 1
b = a - 1;         // 0s with 1s at positions 0 through c1 - 1
n = n | b;         // inserts 1s at positions 0 through c1 - 1
```

Or, more concisely:

```
n |= (1 << (c1 - 1)) - 1;
```

We have now arrived at the smallest number bigger than n with the same number of ones.

The code for `getNext` is below.

```
1  int getNext(int n) {
2      /* Compute c0 and c1 */
3      int c = n;
4      int c0 = 0;
5      int c1 = 0;
6      while (((c & 1) == 0) && (c != 0)) {
7          c0++;
8          c >>= 1;
9      }
10
11     while ((c & 1) == 1) {
12         c1++;
13         c >>= 1;
14     }
15
16     /* Error: if n == 11..1100...00, then there is no bigger number with the same
17      * number of 1s. */
18     if (c0 + c1 == 31 || c0 + c1 == 0) {
19         return -1;
20     }
21
22     int p = c0 + c1; // position of rightmost non-trailing zero
23
24     n |= (1 << p); // Flip rightmost non-trailing zero
25     n &= ~(1 << p) - 1; // Clear all bits to the right of p
26     n |= (1 << (c1 - 1)) - 1; // Insert (c1-1) ones on the right.
```



```

27     return n;
28 }

```

Bit Manipulation Approach for Get Previous Number

To implement `getPrev`, we follow a very similar approach.

1. Compute `c0` and `c1`. Note that `c1` is the number of trailing ones, and `c0` is the size of the block of zeros immediately to the left of the trailing ones.
2. Flip the rightmost non-trailing one to a zero. This will be at position $p = c1 + c0$.
3. Clear all bits to the right of bit p .
4. Insert `c1 + 1` ones immediately to the right of position p .

Note that Step 2 sets bit p to a zero and Step 3 sets bits 0 through $p - 1$ to a zero. We can merge these steps.

Let's walk through this with an example.

Step 1: Initial Number. $p = 7$. $c1 = 2$. $c0 = 5$.

1	0	0	1	1	1	1	0	0	0	0	0	1	1
13	12	11	10	9	8	7	6	5	4	3	2	1	0

Steps 2 & 3: Clear bits 0 through p .

1	0	0	1	1	1	0	0	0	0	0	0	0	0
13	12	11	10	9	8	7	6	5	4	3	2	1	0

We can do this as follows:

```

int a = ~0;           // Sequence of 1s
int b = a << (p + 1); // Sequence of 1s followed by p + 1 zeros.
n &= b;               // Clears bits 0 through p.

```

Steps 4: Insert $c1 + 1$ ones immediately to the right of position p .

1	0	0	1	1	1	0	1	1	1	0	0	0	0
13	12	11	10	9	8	7	6	5	4	3	2	1	0

Note that since $p = c1 + c0$, the $(c1 + 1)$ ones will be followed by $(c0 - 1)$ zeros.

We can do this as follows:

```

int a = 1 << (c1 + 1); // 0s with 1 at position (c1 + 1)
int b = a - 1;         // 0s followed by c1 + 1 ones
int c = b << (c0 - 1); // c1+1 ones followed by c0-1 zeros.
n |= c;

```

The code to implement this is below.

```

1 int getPrev(int n) {
2     int temp = n;
3     int c0 = 0;
4     int c1 = 0;
5     while (temp & 1 == 1) {
6         c1++;
7         temp >>= 1;
8     }
9

```

```

10  if (temp == 0) return -1;
11
12  while (((temp & 1) == 0) && (temp != 0)) {
13      c0++;
14      temp >>= 1;
15  }
16
17  int p = c0 + c1; // position of rightmost non-trailing one
18  n &= ((~0) << (p + 1)); // clears from bit p onwards
19
20  int mask = (1 << (c1 + 1)) - 1; // Sequence of (c1+1) ones
21  n |= mask << (c0 - 1);
22
23  return n;
24 }

```

Arithmetic Approach to Get Next Number

If c_0 is the number of trailing zeros, c_1 is the size of the one block immediately following, and $p = c_0 + c_1$, we can word our solution from earlier as follows:

1. Set the p th bit to 1.
2. Set all bits following p to 0.
3. Set bits 0 through $c_1 - 2$ to 1. This will be $c_1 - 1$ total bits.

A quick and dirty way to perform steps 1 and 2 is to set the trailing zeros to 1 (giving us p trailing ones), and then add 1. Adding one will flip all trailing ones, so we wind up with a 1 at bit p followed by p zeros. We can perform this arithmetically.

```

n += 2c0 - 1; // Sets trailing 0s to 1, giving us p trailing 1s
n += 1;       // Flips first p 1s to 0s, and puts a 1 at bit p.

```

Now, to perform Step 3 arithmetically, we just do:

```

n += 2c1-1 - 1; // Sets trailing c1 - 1 zeros to ones.

```

This math reduces to:

```

next = n + (2c0 - 1) + 1 + (2c1-1 - 1)
      = n + 2c0 + 2c1-1 - 1

```

The best part is that, using a little bit manipulation, it's simple to code.

```

1  int getNextArith(int n) {
2      /* ... same calculation for c0 and c1 as before ... */
3      return n + (1 << c0) + (1 << (c1 - 1)) - 1;
4  }

```

Arithmetic Approach to Get Previous Number

If c_1 is the number of trailing ones, c_0 is the size of the zero block immediately following, and $p = c_0 + c_1$, we can word the initial `getPrev` solution as follows:

1. Set the p th bit to 0
2. Set all bits following p to 1
3. Set bits 0 through $c_0 - 1$ to 0.

We can implement this arithmetically as follows. For clarity in the example, we will assume $n = 10000011$. This makes $c_1 = 2$ and $c_0 = 5$.

```

n -= 2c1 - 1;      // Removes trailing 1s. n is now 10000000.
n -= 1;            // Flips trailing 0s. n is now 01111111.
n -= 2c0-1 - 1;    // Flips last (c0-1) 0s. n is now 01110000.

```

This reduces mathematically to:

```

next = n - (2c1 - 1) - 1 - (2c0-1 - 1).
      = n - 2c1 - 2c0-1 + 1

```

Again, this is very easy to implement.

```

1 int getPrevArith(int n) {
2     /* ... same calculation for c0 and c1 as before ... */
3     return n - (1 << c1) - (1 << (c0 - 1)) + 1;
4 }

```

Whew! Don't worry, you wouldn't be expected to get all this in an interview—at least not without a lot of help from the interviewer.

5.5 Debugger: Explain what the following code does: $((n \& (n-1)) == 0)$.

pg 116

SOLUTION

We can work backwards to solve this question.

What does it mean if $A \& B == 0$?

It means that A and B never have a 1 bit in the same place. So if $n \& (n-1) == 0$, then n and n-1 never share a 1.

What does n-1 look like (as compared with n)?

Try doing subtraction by hand (in base 2 or 10). What happens?

1101011000 [base 2]	593100 [base 10]
- 1	- 1
= 1101010111 [base 2]	= 593099 [base 10]

When you subtract 1 from a number, you look at the least significant bit. If it's a 1 you change it to 0, and you are done. If it's a zero, you must "borrow" from a larger bit. So, you go to increasingly larger bits, changing each bit from a 0 to a 1, until you find a 1. You flip that 1 to a 0 and you are done.

Thus, n-1 will look like n, except that n's initial 0s will be 1s in n-1, and n's least significant 1 will be a 0 in n-1. That is:

```

if      n = abcde1000
then n-1 = abcde0111

```

So what does $n \& (n-1) == 0$ indicate?

n and n-1 must have no 1s in common. Given that they look like this:

```

if      n = abcde1000
then n-1 = abcde0111

```

abcde must be all 0s, which means that n must look like this: 00001000. The value n is therefore a power of two.

So, we have our answer: $((n \& (n-1)) == 0)$ checks if n is a power of 2 (or if n is 0).

5.6 Conversion: Write a function to determine the number of bits you would need to flip to convert integer A to integer B.

EXAMPLE

Input: 29 (or: 11101), 15 (or: 01111)

Output: 2

pg 116

SOLUTION

This seemingly complex problem is actually rather straightforward. To approach this, ask yourself how you would figure out which bits in two numbers are different. Simple: with an XOR.

Each 1 in the XOR represents a bit that is different between A and B. Therefore, to check the number of bits that are different between A and B, we simply need to count the number of bits in $A \oplus B$ that are 1.

```
1 int bitSwapRequired(int a, int b) {
2     int count = 0;
3     for (int c = a ^ b; c != 0; c = c >> 1) {
4         count += c & 1;
5     }
6     return count;
7 }
```

This code is good, but we can make it a bit better. Rather than simply shifting c repeatedly while checking the least significant bit, we can continuously flip the least significant bit and count how long it takes c to reach 0. The operation $c = c \& (c - 1)$ will clear the least significant bit in c .

The code below utilizes this approach.

```
1 int bitSwapRequired(int a, int b) {
2     int count = 0;
3     for (int c = a ^ b; c != 0; c = c & (c-1)) {
4         count++;
5     }
6     return count;
7 }
```

The above code is one of those bit manipulation problems that comes up sometimes in interviews. Though it'd be hard to come up with it on the spot if you've never seen it before, it is useful to remember the trick for your interviews.

5.7 Pairwise Swap: Write a program to swap odd and even bits in an integer with as few instructions as possible (e.g., bit 0 and bit 1 are swapped, bit 2 and bit 3 are swapped, and so on).

pg 116

SOLUTION

Like many of the previous problems, it's useful to think about this problem in a different way. Operating on individual pairs of bits would be difficult, and probably not that efficient either. So how else can we think about this problem?

We can approach this as operating on the odds bits first, and then the even bits. Can we take a number n and move the odd bits over by 1? Sure. We can mask all odd bits with 10101010 in binary (which is 0xAA),

then shift them right by 1 to put them in the even spots. For the even bits, we do an equivalent operation. Finally, we merge these two values.

This takes a total of five instructions. The code below implements this approach.

```
1  int swapOddEvenBits(int x) {
2      return ( ((x & 0xaaaaaaaa) >>> 1) | ((x & 0x55555555) << 1) );
3  }
```

Note that we use the logical right shift, instead of the arithmetic right shift. This is because we want the sign bit to be filled with a zero.

We've implemented the code above for 32-bit integers in Java. If you were working with 64-bit integers, you would need to change the mask. The logic, however, would remain the same.

5.8 Draw Line: A monochrome screen is stored as a single array of bytes, allowing eight consecutive pixels to be stored in one byte. The screen has width *w*, where *w* is divisible by 8 (that is, no byte will be split across rows). The height of the screen, of course, can be derived from the length of the array and the width. Implement a function that draws a horizontal line from (*x*₁, *y*) to (*x*₂, *y*).

The method signature should look something like:

```
drawLine(byte[] screen, int width, int x1, int x2, int y)
```

pg 116

SOLUTION

A naive solution to the problem is straightforward: iterate in a for loop from *x*₁ to *x*₂, setting each pixel along the way. But that's hardly any fun, is it? (Nor is it very efficient.)

A better solution is to recognize that if *x*₁ and *x*₂ are far away from each other, several full bytes will be contained between them. These full bytes can be set one at a time by doing `screen[byte_pos] = 0xFF`. The residual start and end of the line can be set using masks.

```
1  void drawLine(byte[] screen, int width, int x1, int x2, int y) {
2      int start_offset = x1 % 8;
3      int first_full_byte = x1 / 8;
4      if (start_offset != 0) {
5          first_full_byte++;
6      }
7
8      int end_offset = x2 % 8;
9      int last_full_byte = x2 / 8;
10     if (end_offset != 7) {
11         last_full_byte--;
12     }
13
14     // Set full bytes
15     for (int b = first_full_byte; b <= last_full_byte; b++) {
16         screen[(width / 8) * y + b] = (byte) 0xFF;
17     }
18
19     // Create masks for start and end of line
20     byte start_mask = (byte) (0xFF >> start_offset);
21     byte end_mask = (byte) ~(0xFF >> (end_offset + 1));
22
23     // Set start and end of line
24     if ((x1 / 8) == (x2 / 8)) { // x1 and x2 are in the same byte
```



```
25     byte mask = (byte) (start_mask & end_mask);
26     screen[(width / 8) * y + (x1 / 8)] |= mask;
27 } else {
28     if (start_offset != 0) {
29         int byte_number = (width / 8) * y + first_full_byte - 1;
30         screen[byte_number] |= start_mask;
31     }
32     if (end_offset != 7) {
33         int byte_number = (width / 8) * y + last_full_byte + 1;
34         screen[byte_number] |= end_mask;
35     }
36 }
37 }
```

Be careful on this problem; there are a lot of “gotchas” and special cases. For example, you need to consider the case where x_1 and x_2 are in the same byte. Only the most careful candidates can implement this code bug-free.

6

Solutions to Math and Logic Puzzles

- 6.1 The Heavy Pill:** You have 20 bottles of pills. 19 bottles have 1.0 gram pills, but one has pills of weight 1.1 grams. Given a scale that provides an exact measurement, how would you find the heavy bottle? You can only use the scale once.

pg 122

SOLUTION

Sometimes, tricky constraints can be a clue. This is the case with the constraint that we can only use the scale once.

Because we can only use the scale once, we know something interesting: we must weigh multiple pills at the same time. In fact, we know we must weigh pills from at least 19 bottles at the same time. Otherwise, if we skipped two or more bottles entirely, how could we distinguish between those missed bottles? Remember that we only have *one* chance to use the scale.

So how can we weigh pills from more than one bottle and discover which bottle has the heavy pills? Let's suppose there were just two bottles, one of which had heavier pills. If we took one pill from each bottle, we would get a weight of 2.1 grams, but we wouldn't know which bottle contributed the extra 0.1 grams. We know we must treat the bottles differently somehow.

If we took one pill from Bottle #1 and two pills from Bottle #2, what would the scale show? It depends. If Bottle #1 were the heavy bottle, we would get 3.1 grams. If Bottle #2 were the heavy bottle, we would get 3.2 grams. And that is the trick to this problem.

We know the "expected" weight of a bunch of pills. The difference between the expected weight and the actual weight will indicate which bottle contributed the heavier pills, *provided* we select a different number of pills from each bottle.

We can generalize this to the full solution: take one pill from Bottle #1, two pills from Bottle #2, three pills from Bottle #3, and so on. Weigh this mix of pills. If all pills were one gram each, the scale would read 210 grams ($1 + 2 + \dots + 20 = 20 * 21 / 2 = 210$). Any "overage" must come from the extra 0.1 gram pills.

This formula will tell you the bottle number:

$$\frac{\text{weight} - 210 \text{ grams}}{0.1 \text{ grams}}$$

So, if the set of pills weighed 211.3 grams, then Bottle #13 would have the heavy pills.

6.2 Basketball: You have a basketball hoop and someone says that you can play one of two games.

Game 1: You get one shot to make the hoop.

Game 2: You get three shots and you have to make two of three shots.

If p is the probability of making a particular shot, for which values of p should you pick one game or the other?

pg 123

SOLUTION

To solve this problem, we can apply straightforward probability laws by comparing the probabilities of winning each game.

Probability of winning Game 1:

The probability of winning Game 1 is p , by definition.

Probability of winning Game 2:

Let $s(k, n)$ be the probability of making exactly k shots out of n . The probability of winning Game 2 is the probability of making exactly two shots out of three OR making all three shots. In other words:

$$P(\text{winning}) = s(2, 3) + s(3, 3)$$

The probability of making all three shots is:

$$s(3, 3) = p^3$$

The probability of making exactly two shots is:

$$\begin{aligned} &P(\text{making 1 and 2, and missing 3}) \\ &\quad + P(\text{making 1 and 3, and missing 2}) \\ &\quad + P(\text{missing 1, and making 2 and 3}) \\ &= p * p * (1 - p) + p * (1 - p) * p + (1 - p) * p * p \\ &= 3(1 - p)p^2 \end{aligned}$$

Adding these together, we get:

$$\begin{aligned} &= p^3 + 3(1 - p)p^2 \\ &= p^3 + 3p^2 - 3p^3 \\ &= 3p^2 - 2p^3 \end{aligned}$$

Which game should you play?

You should play Game 1 if $P(\text{Game 1}) > P(\text{Game 2})$:

$$\begin{aligned} p &> 3p^2 - 2p^3 \\ 1 &> 3p - 2p^2 \\ 2p^2 - 3p + 1 &> 0 \\ (2p - 1)(p - 1) &> 0 \end{aligned}$$

Both terms must be positive, or both must be negative. But we know $p < 1$, so $p - 1 < 0$. This means both terms must be negative.

$$\begin{aligned} 2p - 1 &< 0 \\ 2p &< 1 \\ p &< .5 \end{aligned}$$

So, we should play Game 1 if $0 < p < .5$ and Game 2 if $.5 < p < 1$.

If $p = 0, 0.5$, or 1 , then $P(\text{Game 1}) = P(\text{Game 2})$, so it doesn't matter which game we play.

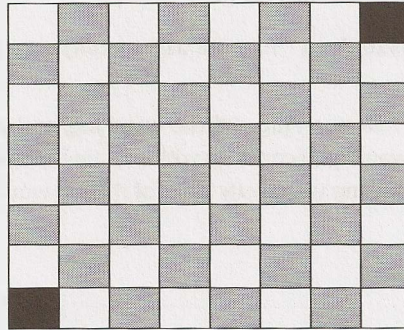
- 6.3 Dominos:** There is an 8x8 chessboard in which two diagonally opposite corners have been cut off. You are given 31 dominos, and a single domino can cover exactly two squares. Can you use the 31 dominos to cover the entire board? Prove your answer (by providing an example or showing why it's impossible).

pg 123

SOLUTION

At first, it seems like this should be possible. It's an 8 x 8 board, which has 64 squares, but two have been cut off, so we're down to 62 squares. A set of 31 dominoes should be able to fit there, right?

When we try to lay down dominoes on row 1, which only has 7 squares, we may notice that one domino must stretch into the row 2. Then, when we try to lay down dominoes onto row 2, again we need to stretch a domino into row 3.



For each row we place, we'll always have one domino that needs to poke into the next row. No matter how many times and ways we try to solve this issue, we won't be able to successfully lay down all the dominoes.

There's a cleaner, more solid proof for why it won't work. The chessboard initially has 32 black and 32 white squares. By removing opposite corners (which must be the same color), we're left with 30 of one color and 32 of the other color. Let's say, for the sake of argument, that we have 30 black and 32 white squares.

Each domino we set on the board will always take up one white and one black square. Therefore, 31 dominos will take up 31 white squares and 31 black squares exactly. On this board, however, we must have 30 black squares and 32 white squares. Hence, it is impossible.

- 6.4 Ants on a Triangle:** There are three ants on different vertices of a triangle. What is the probability of collision (between any two or all of them) if they start walking on the sides of the triangle? Assume that each ant randomly picks a direction, with either direction being equally likely to be chosen, and that they walk at the same speed.

Similarly, find the probability of collision with n ants on an n -vertex polygon.

pg 123

SOLUTION

The ants will collide if any of them are moving towards each other. So, the only way that they won't collide is if they are all moving in the same direction (clockwise or counterclockwise). We can compute this probability and work backwards from there.

Since each ant can move in two directions, and there are three ants, the probability is:

$$P(\text{clockwise}) = (\frac{1}{2})^3$$
$$P(\text{counter clockwise}) = (\frac{1}{2})^3$$
$$P(\text{same direction}) = (\frac{1}{2})^3 + (\frac{1}{2})^3 = \frac{1}{4}$$

The probability of collision is therefore the probability of the ants *not* moving in the same direction:

$$P(\text{collision}) = 1 - P(\text{same direction}) = 1 - \frac{1}{4} = \frac{3}{4}$$

To generalize this to an n-vertex polygon: there are still only two ways in which the ants can move to avoid a collision, but there are 2ⁿ ways they can move in total. Therefore, in general, probability of collision is:

$$P(\text{clockwise}) = (\frac{1}{2})^n$$
$$P(\text{counter}) = (\frac{1}{2})^n$$
$$P(\text{same direction}) = 2 (\frac{1}{2})^n = (\frac{1}{2})^{n-1}$$
$$P(\text{collision}) = 1 - P(\text{same direction}) = 1 - (\frac{1}{2})^{n-1}$$

6.5 Jugs of Water: You have a five-quart jug, a three-quart jug, and an unlimited supply of water (but no measuring cups). How would you come up with exactly four quarts of water? Note that the jugs are oddly shaped, such that filling up exactly “half” of the jug would be impossible.

pg 123

SOLUTION

If we just play with the jugs, we'll find that we can pour water back and forth between them as follows:

5 Quart	3 Quart	Action
5	0	Filled 5-quart jug.
2	3	Filled 3-quart with 5-quart's contents.
2	0	Dumped 3-quart.
0	2	Fill 3-quart with 5-quart's contents.
5	2	Filled 5-quart.
4	3	Fill remainder of 3-quart with 5-quart.
4		Done! We have 4 quarts.

This question, like many puzzle questions, has a math/computer science root. If the two jug sizes are relatively prime, you can measure any value between one and the sum of the jug sizes.

- 6.6 Blue-Eyed Island:** A bunch of people are living on an island, when a visitor comes with a strange order: *all blue-eyed people must leave the island as soon as possible*. There will be a flight out at 8:00pm every evening. Each person can see everyone else's eye color, but they do not know their own (nor is anyone allowed to tell them). Additionally, they do not know how many people have blue eyes, although they do know that at least one person does. How many days will it take the blue-eyed people to leave?

pg 123

SOLUTION

Let's apply the Base Case and Build approach. Assume that there are n people on the island and c of them have blue eyes. We are explicitly told that $c > 0$.

Case $c = 1$: Exactly one person has blue eyes.

Assuming all the people are intelligent, the blue-eyed person should look around and realize that no one else has blue eyes. Since he knows that at least one person has blue eyes, he must conclude that it is he who has blue eyes. Therefore, he would take the flight that evening.

Case $c = 2$: Exactly two people have blue eyes.

The two blue-eyed people see each other, but are unsure whether c is 1 or 2. They know, from the previous case, that if $c = 1$, the blue-eyed person would leave on the first night. Therefore, if the other blue-eyed person is still there, he must deduce that $c = 2$, which means that he himself has blue eyes. Both men would then leave on the second night.

Case $c > 2$: The General Case.

As we increase c , we can see that this logic continues to apply. If $c = 3$, then those three people will immediately know that there are either 2 or 3 people with blue eyes. If there were two people, then those two people would have left on the second night. So, when the others are still around after that night, each person would conclude that $c = 3$ and that they, therefore, have blue eyes too. They would leave that night.

This same pattern extends up through any value of c . Therefore, if c men have blue eyes, it will take c nights for the blue-eyed men to leave. All will leave on the same night.

- 6.7 The Apocalypse:** In the new post-apocalyptic world, the world queen is desperately concerned about the birth rate. Therefore, she decrees that all families should ensure that they have one girl or else they face massive fines. If all families abide by this policy—that is, they have continue to have children until they have one girl, at which point they immediately stop—what will the gender ratio of the new generation be? (Assume that the odds of someone having a boy or a girl on any given pregnancy is equal.) Solve this out logically and then write a computer simulation of it.

pg 123

SOLUTION

If each family abides by this policy, then each family will have a sequence of zero or more boys followed by a single girl. That is, if "G" indicates a girl and "B" indicates a boy, the sequence of children will look like one of: G; BG; BBG; BBBG; BBBBG; and so on.

We can solve this problem multiple ways.

Mathematically

We can work out the probability for each gender sequence.

- $P(G) = \frac{1}{2}$. That is, 50% of families will have a girl first. The others will go on to have more children.
- $P(BG) = \frac{1}{4}$. Of those who have a second child (which is 50%), 50% of them will have a girl the next time.
- $P(BBG) = \frac{1}{8}$. Of those who have a third child (which is 25%), 50% of them will have a girl the next time.

And so on.

We know that every family has exactly one girl. How many boys does each family have, on average? To compute this, we can look at the expected value of the number of boys. The expected value of the number of boys is the probability of each sequence multiplied by the number of boys in that sequence.

Sequence	Number of Boys	Probability	Number of Boys * Probability
G	0	$\frac{1}{2}$	0
BG	1	$\frac{1}{4}$	$\frac{1}{4}$
BBG	2	$\frac{1}{8}$	$\frac{2}{8}$
BBBG	3	$\frac{1}{16}$	$\frac{3}{16}$
BBBBG	4	$\frac{1}{32}$	$\frac{4}{32}$
BBBBBG	5	$\frac{1}{64}$	$\frac{5}{64}$
BBBBBBG	6	$\frac{1}{128}$	$\frac{6}{128}$

Or in other words, this is the sum of i to infinity of i divided by 2^i .

$$\sum_{i=0}^{\infty} \frac{i}{2^i}$$

You probably won't know this off the top of your head, but we can try to estimate it. Let's try converting the above values to a common denominator of 128 (2^6).

$$\frac{1}{4} = \frac{32}{128}$$

$$\frac{4}{32} = \frac{16}{128}$$

$$\frac{2}{8} = \frac{32}{128}$$

$$\frac{5}{64} = \frac{10}{128}$$

$$\frac{3}{16} = \frac{24}{128}$$

$$\frac{6}{128} = \frac{6}{128}$$

$$\frac{32 + 32 + 24 + 16 + 10 + 6}{128} = \frac{120}{128}$$

This looks like it's going to inch closer to $\frac{128}{128}$ (which is of course 1). This "looks like" intuition is valuable, but it's not exactly a mathematical concept. It's a clue though and we can turn to logic here. Should it be 1?

Logically

If the earlier sum is 1, this would mean that the gender ratio is even. Families contribute exactly one girl and on average one boy. The birth policy is therefore ineffective. Does this make sense?

At first glance, this seems wrong. The policy is designed to favor girls as it ensures that all families have a girl.

On the other hand, the families that keep having children contribute (potentially) multiple boys to the population. This could offset the impact of the “one girl” policy.

One way to think about this is to imagine that we put all the gender sequence of each family into one giant string. So if family 1 has BG, family 2 has BBG, and family 3 has G, we would write BGBBGG.

In fact, we don’t really care about the groupings of families because we’re concerned about the population as a whole. As soon as a child is born, we can just append its gender (B or G) to the string.

What are the odds of the next character being a G? Well, if the odds of having a boy and girl is the same, then the odds of the next character being a G is 50%. Therefore, roughly half of the string should be Gs and half should be Bs, giving an even gender ratio.

This actually makes a lot of sense. Biology hasn’t been changed. Half of newborn babies are girls and half are boys. Abiding by some rule about when to stop having children doesn’t change this fact.

Therefore, the gender ratio is 50% girls and 50% boys.

Simulation

We’ll write this in a simple way that directly corresponds to the problem.

```

1  double runNFamilies(int n) {
2      int boys = 0;
3      int girls = 0;
4      for (int i = 0; i < n; i++) {
5          int[] genders = runOneFamily();
6          girls += genders[0];
7          boys += genders[1];
8      }
9      return girls / (double) (boys + girls);
10 }
11
12 int[] runOneFamily() {
13     Random random = new Random();
14     int boys = 0;
15     int girls = 0;
16     while (girls == 0) { // until we have a girl
17         if (random.nextBoolean()) { // girl
18             girls += 1;
19         } else { // boy
20             boys += 1;
21         }
22     }
23     int[] genders = {girls, boys};
24     return genders;
25 }
```

Sure enough, if you run this on large values of n , you should get something very close to 0.5.

6.8 The Egg Drop Problem: There is a building of 100 floors. If an egg drops from the Nth floor or above, it will break. If it's dropped from any floor below, it will not break. You're given two eggs. Find N, while minimizing the number of drops for the worst case.

pg 124

SOLUTION

We may observe that, regardless of how we drop Egg 1, Egg 2 must do a linear search (from lowest to highest) between the "breaking floor" and the next highest non-breaking floor. For example, if Egg 1 is dropped from floors 5 and 10 without breaking, but it breaks when it's dropped from floor 15, then Egg 2 must be dropped, in the worst case, from floors 11, 12, 13, and 14.

The Approach

As a first try, suppose we drop an egg from the 10th floor, then the 20th, ...

- If Egg 1 breaks on the first drop (floor 10), then we have at most 10 drops total.
- If Egg 1 breaks on the last drop (floor 100), then we have at most 19 drops total (floors 10, 20, ..., 90, 100, then 91 through 99).

That's pretty good, but all we've considered is the absolute worst case. We should do some "load balancing" to make those two cases more even.

Our goal is to create a system for dropping Egg 1 such that the number of drops is as consistent as possible, whether Egg 1 breaks on the first drop or the last drop.

1. A perfectly load-balanced system would be one in which $\text{Drops}(\text{Egg } 1) + \text{Drops}(\text{Egg } 2)$ is always the same, regardless of where Egg 1 breaks.
2. For that to be the case, since each drop of Egg 1 takes one more step, Egg 2 is allowed one fewer step.
3. We must, therefore, reduce the number of steps potentially required by Egg 2 by one drop each time. For example, if Egg 1 is dropped on floor 20 and then floor 30, Egg 2 is potentially required to take 9 steps. When we drop Egg 1 again, we must reduce potential Egg 2 steps to only 8. That is, we must drop Egg 1 at floor 39.
4. Therefore, Egg 1 must start at floor X, then go up by X-1 floors, then X-2, ..., until it gets to 100.
5. Solve for X.

$$X + (X - 1) + (X - 2) + \dots + 1 = 100$$

$$\frac{X(X+1)}{2} = 100$$

$$X \approx 13.65$$

X clearly needs to be an integer. Should we round X up or down?

- If we round X up to 14, then we would go up by 14, then 13, then 12, and so on. The last increment would be 4, and it would happen on floor 99. If Egg 1 broke on any of the prior floors, we know we've balanced the eggs such that the number of drops of Egg 1 and Egg 2 always sum to the same thing: 14. If Egg 1 hasn't broken by floor 99, then we just need one more drop to determine if it will break at floor 100. Either way, the number of drops is no more than 14.
- If we round X down to 13, then we would go up by 13, then 12, then 11, and so on. The last increment will be 1 and it will happen at floor 91. This is after 13 drops. Floors 92 through 100 have not been covered yet. We can't cover those floors in just one drop (which would be necessary to merely tie the

“round up” case).

Therefore, we should round X up to 14. That is, we go to floor 14, then 27, then 39, This takes 14 steps in the worse case.

As in many other maximizing / minimizing problems, the key in this problem is “worst case balancing.”

The following code simulates this approach.

```

1  int breakingPoint = ...;
2  int countDrops = 0;
3
4  boolean drop(int floor) {
5      countDrops++;
6      return floor >= breakingPoint;
7  }
8
9  int findBreakingPoint(int floors) {
10     int interval = 14;
11     int previousFloor = 0;
12     int egg1 = interval;
13
14     /* Drop egg1 at decreasing intervals. */
15     while (!drop(egg1) && egg1 <= floors) {
16         interval -= 1;
17         previousFloor = egg1;
18         egg1 += interval;
19     }
20
21     /* Drop egg2 at 1 unit increments. */
22     int egg2 = previousFloor + 1;
23     while (egg2 < egg1 && egg2 <= floors && !drop(egg2)) {
24         egg2 += 1;
25     }
26
27     /* If it didn't break, return -1. */
28     return egg2 > floors ? -1 : egg2;
29 }

```

If we want to generalize this code for more building sizes, then we can solve for x in:

$$\frac{x(x+1)}{2} = \text{number of floors}$$

This will involve the quadratic formula.

6.9 100 Lockers: There are 100 closed lockers in a hallway. A man begins by opening all 100 lockers. Next, he closes every second locker. Then, on his third pass, he toggles every third locker (closes it if it is open or opens it if it is closed). This process continues for 100 passes, such that on each pass i , the man toggles every i th locker. After his 100th pass in the hallway, in which he toggles only locker #100, how many lockers are open?

pg 124

SOLUTION

We can tackle this problem by thinking through what it means for a door to be toggled. This will help us deduce which doors at the very end will be left opened.

Question: For which rounds is a door toggled (open or closed)?

A door n is toggled once for each factor of n , including itself and 1. That is, door 15 is toggled on rounds 1, 3, 5, and 15.

Question: When would a door be left open?

A door is left open if the number of factors (which we will call x) is odd. You can think about this by pairing factors off as an open and a close. If there's one remaining, the door will be open.

Question: When would x be odd?

The value x is odd if n is a perfect square. Here's why: pair n 's factors by their complements. For example, if n is 36, the factors are (1, 36), (2, 18), (3, 12), (4, 9), (6, 6). Note that (6, 6) only contributes one factor, thus giving n an odd number of factors.

Question: How many perfect squares are there?

There are 10 perfect squares. You could count them (1, 4, 9, 16, 25, 36, 49, 64, 81, 100), or you could simply realize that you can take the numbers 1 through 10 and square them:

$$1*1, 2*2, 3*3, \dots, 10*10$$

Therefore, there are 10 lockers open at the end of this process.

6.10 Poison: You have 1000 bottles of soda, and exactly one is poisoned. You have 10 test strips which can be used to detect poison. A single drop of poison will turn the test strip positive permanently. You can put any number of drops on a test strip at once and you can reuse a test strip as many times as you'd like (as long as the results are negative). However, you can only run tests once per day and it takes seven days to return a result. How would you figure out the poisoned bottle in as few days as possible?

Follow up: Write code to simulate your approach.

pg 124

SOLUTION

Observe the wording of the problem. Why seven days? Why not have the results just return immediately?

The fact that there's such a lag between starting a test and reading the results likely means that we'll be doing something else in the meantime (running additional tests). Let's hold on to that thought, but start off with a simple approach just to wrap our heads around the problem.

Naive Approach (28 days)

A simple approach is to divide the bottles across the 10 test strips, first in groups of 100. Then, we wait seven days. When the results come back, we look for a positive result across the test strips. We select the bottles associated with the positive test strip, "toss" (i.e., ignore) all the other bottles, and repeat the process. We perform this operation until there is only one bottle left in the test set.

1. Divide bottles across available test strips, one drop per test strip.
2. After seven days, check the test strips for results.
3. On the positive test strip: select the bottles associated with it into a new set of bottles. If this set size is 1,

we have located the poisoned bottle. If it's greater than one, go to step 1.

To simulate this, we'll build classes for `Bottle` and `TestStrip` that mirror the problem's functionality.

```

1  class Bottle {
2      private boolean poisoned = false;
3      private int id;
4
5      public Bottle(int id) { this.id = id; }
6      public int getId() { return id; }
7      public void setAsPoisoned() { poisoned = true; }
8      public boolean isPoisoned() { return poisoned; }
9  }
10
11 class TestStrip {
12     public static int DAYS_FOR_RESULT = 7;
13     private ArrayList<ArrayList<Bottle>> dropsByDay =
14         new ArrayList<ArrayList<Bottle>>();
15     private int id;
16
17     public TestStrip(int id) { this.id = id; }
18     public int getId() { return id; }
19
20     /* Resize list of days/drops to be large enough. */
21     private void sizeDropsForDay(int day) {
22         while (dropsByDay.size() <= day) {
23             dropsByDay.add(new ArrayList<Bottle>());
24         }
25     }
26
27     /* Add drop from bottle on specific day. */
28     public void addDropOnDay(int day, Bottle bottle) {
29         sizeDropsForDay(day);
30         ArrayList<Bottle> drops = dropsByDay.get(day);
31         drops.add(bottle);
32     }
33
34     /* Checks if any of the bottles in the set are poisoned. */
35     private boolean hasPoison(ArrayList<Bottle> bottles) {
36         for (Bottle b : bottles) {
37             if (b.isPoisoned()) {
38                 return true;
39             }
40         }
41         return false;
42     }
43
44     /* Gets bottles used in the test DAYS_FOR_RESULT days ago. */
45     public ArrayList<Bottle> getLastWeeksBottles(int day) {
46         if (day < DAYS_FOR_RESULT) {
47             return null;
48         }
49         return dropsByDay.get(day - DAYS_FOR_RESULT);
50     }
51
52     /* Checks for poisoned bottles since before DAYS_FOR_RESULT */
53     public boolean isPositiveOnDay(int day) {

```

```

54     int testDay = day - DAYS_FOR_RESULT;
55     if (testDay < 0 || testDay >= dropsByDay.size()) {
56         return false;
57     }
58     for (int d = 0; d <= testDay; d++) {
59         ArrayList<Bottle> bottles = dropsByDay.get(d);
60         if (hasPoison(bottles)) {
61             return true;
62         }
63     }
64     return false;
65 }
66 }

```

This is just one way of simulating the behavior of the bottles and test strips, and each has its pros and cons.

With this infrastructure built, we can now implement code to test our approach.

```

1  int findPoisonedBottle(ArrayList<Bottle> bottles, ArrayList<TestStrip> strips) {
2      int today = 0;
3
4      while (bottles.size() > 1 && strips.size() > 0) {
5          /* Run tests. */
6          runTestSet(bottles, strips, today);
7
8          /* Wait for results. */
9          today += TestStrip.DAYS_FOR_RESULT;
10
11         /* Check results. */
12         for (TestStrip strip : strips) {
13             if (strip.isPositiveOnDay(today)) {
14                 bottles = strip.getLastWeeksBottles(today);
15                 strips.remove(strip);
16                 break;
17             }
18         }
19     }
20
21     if (bottles.size() == 1) {
22         return bottles.get(0).getId();
23     }
24     return -1;
25 }
26
27 /* Distribute bottles across test strips evenly. */
28 void runTestSet(ArrayList<Bottle> bottles, ArrayList<TestStrip> strips, int day) {
29     int index = 0;
30     for (Bottle bottle : bottles) {
31         TestStrip strip = strips.get(index);
32         strip.addDropOnDay(day, bottle);
33         index = (index + 1) % strips.size();
34     }
35 }
36
37 /* The complete code can be found in the downloadable code attachment. */

```

Note that this approach makes the assumption that there will always be multiple test strips at each round. This assumption is valid for 1000 bottles and 10 test strips.

If we can't assume this, we can implement a fail-safe. If we have just one test strip remaining, we start doing one bottle at a time: test a bottle, wait a week, test another bottle. This approach will take at most 28 days.

Optimized Approach (10 days)

As noted in the beginning of the solution, it might be more optimal to run multiple tests at once.

If we divide the bottles up into 10 groups (with bottles 0 - 99 going to strip 0, bottles 100 - 199 going to strip 1, bottles 200 - 299 going to strip 2, and so on), then day 7 will reveal the first digit of the bottle number. A positive result on strip 1 at day 7 shows that the first digit (100's digit) of the bottle number is 1.

Dividing the bottles in a different way can reveal the second or third digit. We just need to run these tests on different days so that we don't confuse the results.

	Day 0 -> 7	Day 1 -> 8	Day 2 -> 9
Strip 0	0xx	x0x	xx0
Strip 1	1xx	x1x	xx1
Strip 2	2xx	x2x	xx2
Strip 3	3xx	x3x	xx3
Strip 4	4xx	x4x	xx4
Strip 5	5xx	x5x	xx5
Strip 6	6xx	x6x	xx6
Strip 7	7xx	x7x	xx7
Strip 8	8xx	x8x	xx8
Strip 9	9xx	x9x	xx9

For example, if day 7 showed a positive result on strip 4, day 8 showed a positive result on strip 3, and day 9 showed a positive result on strip 8, then this would map to bottle #438.

This mostly works, except for one edge case: what happens if the poisoned bottle has a duplicate digit? For example, bottle #882 or bottle #383.

In fact, these cases are quite different. If day 8 doesn't have any "new" positive results, then we can conclude that digit 2 equals digit 1.

The bigger issue is what happens if day 9 doesn't have any new positive results. In this case, all we know is that digit 3 equals either digit 1 or digit 2. We could not distinguish between bottle #383 and bottle #388. They will both have the same pattern of test results.

We will need to run one additional test. We could run this at the end to clear up ambiguity, but we can also run it at day 3, just in case there's any ambiguity. All we need to do is shift the final digit so that it winds up in a different place than day 2's results.

	Day 0 -> 7	Day 1 -> 8	Day 2 -> 9	Day 3 -> 10
Strip 0	0xx	x0x	xx0	xx9
Strip 1	1xx	x1x	xx1	xx0
Strip 2	2xx	x2x	xx2	xx1
Strip 3	3xx	x3x	xx3	xx2
Strip 4	4xx	x4x	xx4	xx3
Strip 5	5xx	x5x	xx5	xx4

	Day 0 -> 7	Day 1 -> 8	Day 2 -> 9	Day 3 -> 10
Strip 6	6xx	x6x	xx6	xx5
Strip 7	7xx	x7x	xx7	xx6
Strip 8	8xx	x8x	xx8	xx7
Strip 9	9xx	x9x	xx9	xx8

Now, bottle #383 will see (Day 7 = #3, Day 8 -> #8, Day 9 -> [NONE], Day 10 -> #4), while bottle #388 will see (Day 7 = #3, Day 8 -> #8, Day 9 -> [NONE], Day 10 -> #9). We can distinguish between these by "reversing" the shifting on day 10's results.

What happens, though, if day 10 still doesn't see any new results? Could this happen?

Actually, yes. Bottle #898 would see (Day 7 = #8, Day 8 -> #9, Day 9 -> [NONE], Day 10 -> [NONE]). That's okay, though. We just need to distinguish bottle #898 from #899. Bottle #899 will see (Day 7 = #8, Day 8 -> #9, Day 9 -> [NONE], Day 10 -> #0).

The "ambiguous" bottles from day 9 will always map to different values on day 10. The logic is:

- If Day 3->10's test reveals a new test result, "unshift" this value to derive the third digit.
- Otherwise, we know that the third digit equals either the first digit or the second digit *and* that the third digit, when shifted, still equals either the first digit or the second digit. Therefore, we just need to figure out whether the first digit "shifts" into the second digit or the other way around. In the former case, the third digit equals the first digit. In the latter case, the third digit equals the second digit.

Implementing this requires some careful work to prevent bugs.

```

1  int findPoisonedBottle(ArrayList<Bottle> bottles, ArrayList<TestStrip> strips) {
2      if (bottles.size() > 1000 || strips.size() < 10) return -1;
3
4      int tests = 4; // three digits, plus one extra
5      int nTestStrips = strips.size();
6
7      /* Run tests. */
8      for (int day = 0; day < tests; day++) {
9          runTestSet(bottles, strips, day);
10     }
11
12     /* Get results. */
13     HashSet<Integer> previousResults = new HashSet<Integer>();
14     int[] digits = new int[tests];
15     for (int day = 0; day < tests; day++) {
16         int resultDay = day + TestStrip.DAYS_FOR_RESULT;
17         digits[day] = getPositiveOnDay(strips, resultDay, previousResults);
18         previousResults.add(digits[day]);
19     }
20
21     /* If day 1's results matched day 0's, update the digit. */
22     if (digits[1] == -1) {
23         digits[1] = digits[0];
24     }
25
26     /* If day 2 matched day 0 or day 1, check day 3. Day 3 is the same as day 2, but
27      * incremented by 1. */
28     if (digits[2] == -1) {

```



```

29     if (digits[3] == -1) { /* Day 3 didn't give new result */
30         /* Digit 2 equals digit 0 or digit 1. But, digit 2, when incremented also
31            * matches digit 0 or digit 1. This means that digit 0 incremented matches
32            * digit 1, or the other way around. */
33         digits[2] = ((digits[0] + 1) % nTestStrips) == digits[1] ?
34             digits[0] : digits[1];
35     } else {
36         digits[2] = (digits[3] - 1 + nTestStrips) % nTestStrips;
37     }
38 }
39
40 return digits[0] * 100 + digits[1] * 10 + digits[2];
41 }
42
43 /* Run set of tests for this day. */
44 void runTestSet(ArrayList<Bottle> bottles, ArrayList<TestStrip> strips, int day) {
45     if (day > 3) return; // only works for 3 days (digits) + one extra
46
47     for (Bottle bottle : bottles) {
48         int index = getTestStripIndexForDay(bottle, day, strips.size());
49         TestStrip testStrip = strips.get(index);
50         testStrip.addDropOnDay(day, bottle);
51     }
52 }
53
54 /* Get strip that should be used on this bottle on this day. */
55 int getTestStripIndexForDay(Bottle bottle, int day, int nTestStrips) {
56     int id = bottle.getId();
57     switch (day) {
58         case 0: return id / 100;
59         case 1: return (id % 100) / 10;
60         case 2: return id % 10;
61         case 3: return (id % 10 + 1) % nTestStrips;
62         default: return -1;
63     }
64 }
65
66 /* Get results that are positive for a particular day, excluding prior results. */
67 int getPositiveOnDay(ArrayList<TestStrip> testStrips, int day,
68     HashSet<Integer> previousResults) {
69     for (TestStrip testStrip : testStrips) {
70         int id = testStrip.getId();
71         if (testStrip.isPositiveOnDay(day) && !previousResults.contains(id)) {
72             return testStrip.getId();
73         }
74     }
75     return -1;
76 }

```

It will take 10 days in the worst case to get a result with this approach.

Optimal Approach (7 days)

We can actually optimize this slightly more, to return a result in just seven days. This is of course the minimum number of days possible.

Notice what each test strip really means. It's a binary indicator for poisoned or unpoisoned. Is it possible to map 1000 keys to 10 binary values such that each key is mapped to a unique configuration of values? Yes, of course. This is what a binary number is.

We can take each bottle number and look at its binary representation. If there's a 1 in the i th digit, then we will add a drop of this bottle's contents to test strip i . Observe that 2^{10} is 1024, so 10 test strips will be enough to handle up to 1024 bottles.

We wait seven days, and then read the results. If test strip i is positive, then set bit i of the result value. Reading all the test strips will give us the ID of the poisoned bottle.

```
1  int findPoisonedBottle(ArrayList<Bottle> bottles, ArrayList<TestStrip> strips) {
2      runTests(bottles, strips);
3      ArrayList<Integer> positive = getPositiveOnDay(strips, 7);
4      return setBits(positive);
5  }
6
7  /* Add bottle contents to test strips */
8  void runTests(ArrayList<Bottle> bottles, ArrayList<TestStrip> testStrips) {
9      for (Bottle bottle : bottles) {
10         int id = bottle.getId();
11         int bitIndex = 0;
12         while (id > 0) {
13             if ((id & 1) == 1) {
14                 testStrips.get(bitIndex).addDropOnDay(0, bottle);
15             }
16             bitIndex++;
17             id >>= 1;
18         }
19     }
20 }
21
22 /* Get test strips that are positive on a particular day. */
23 ArrayList<Integer> getPositiveOnDay(ArrayList<TestStrip> testStrips, int day) {
24     ArrayList<Integer> positive = new ArrayList<Integer>();
25     for (TestStrip testStrip : testStrips) {
26         int id = testStrip.getId();
27         if (testStrip.isPositiveOnDay(day)) {
28             positive.add(id);
29         }
30     }
31     return positive;
32 }
33
34 /* Create number by setting bits with indices specified in positive. */
35 int setBits(ArrayList<Integer> positive) {
36     int id = 0;
37     for (Integer bitIndex : positive) {
38         id |= 1 << bitIndex;
39     }
40     return id;
41 }
```

This approach will work as long as $2^T \geq B$, where T is the number of test strips and B is the number of bottles.

7

Solutions to Object-Oriented Design

7.1 Deck of Cards: Design the data structures for a generic deck of cards. Explain how you would subclass the data structures to implement blackjack.

pg 127

SOLUTION

First, we need to recognize that a “generic” deck of cards can mean many things. Generic could mean a standard deck of cards that can play a poker-like game, or it could even stretch to Uno or Baseball cards. It is important to ask your interviewer what she means by generic.

Let’s assume that your interviewer clarifies that the deck is a standard 52-card set, like you might see used in a blackjack or poker game. If so, the design might look like this:

```
1  public enum Suit {
2      Club (0), Diamond (1), Heart (2), Spade (3);
3      private int value;
4      private Suit(int v) { value = v; }
5      public int getValue() { return value; }
6      public static Suit getSuitFromValue(int value) { ... }
7  }
8
9  public class Deck <T extends Card> {
10     private ArrayList<T> cards; // all cards, dealt or not
11     private int dealtIndex = 0; // marks first undealt card
12
13     public void setDeckOfCards(ArrayList<T> deckOfCards) { ... }
14
15     public void shuffle() { ... }
16     public int remainingCards() {
17         return cards.size() - dealtIndex;
18     }
19     public T[] dealHand(int number) { ... }
20     public T dealCard() { ... }
21 }
22
23 public abstract class Card {
24     private boolean available = true;
25
26     /* number or face that's on card - a number 2 through 10, or 11 for Jack, 12 for
27      * Queen, 13 for King, or 1 for Ace */
28     protected int faceValue;
29     protected Suit suit;
```

```

30
31 public Card(int c, Suit s) {
32     faceValue = c;
33     suit = s;
34 }
35
36 public abstract int value();
37 public Suit suit() { return suit; }
38
39 /* Checks if the card is available to be given out to someone */
40 public boolean isAvailable() { return available; }
41 public void markUnavailable() { available = false; }
42 public void markAvailable() { available = true; }
43 }
44
45 public class Hand <T extends Card> {
46     protected ArrayList<T> cards = new ArrayList<T>();
47
48     public int score() {
49         int score = 0;
50         for (T card : cards) {
51             score += card.value();
52         }
53         return score;
54     }
55
56     public void addCard(T card) {
57         cards.add(card);
58     }
59 }

```

In the above code, we have implemented Deck with generics but restricted the type of T to Card. We have also implemented Card as an abstract class, since methods like `value()` don't make much sense without a specific game attached to them. (You could make a compelling argument that they should be implemented anyway, by defaulting to standard poker rules.)

Now, let's say we're building a blackjack game, so we need to know the value of the cards. Face cards are 10 and an ace is 11 (most of the time, but that's the job of the Hand class, not the following class).

```

1 public class BlackJackHand extends Hand<BlackJackCard> {
2     /* There are multiple possible scores for a blackjack hand, since aces have
3      * multiple values. Return the highest possible score that's under 21, or the
4      * lowest score that's over. */
5     public int score() {
6         ArrayList<Integer> scores = possibleScores();
7         int maxUnder = Integer.MIN_VALUE;
8         int minOver = Integer.MAX_VALUE;
9         for (int score : scores) {
10             if (score > 21 && score < minOver) {
11                 minOver = score;
12             } else if (score <= 21 && score > maxUnder) {
13                 maxUnder = score;
14             }
15         }
16         return maxUnder == Integer.MIN_VALUE ? minOver : maxUnder;
17     }
18 }

```

```

19  /* return a list of all possible scores this hand could have (evaluating each
20   * ace as both 1 and 11 */
21  private ArrayList<Integer> possibleScores() { ... }
22
23  public boolean busted() { return score() > 21; }
24  public boolean is21() { return score() == 21; }
25  public boolean isBlackJack() { ... }
26 }
27
28 public class BlackJackCard extends Card {
29     public BlackJackCard(int c, Suit s) { super(c, s); }
30     public int value() {
31         if (isAce()) return 1;
32         else if (faceValue >= 11 && faceValue <= 13) return 10;
33         else return faceValue;
34     }
35
36     public int minValue() {
37         if (isAce()) return 1;
38         else return value();
39     }
40
41     public int maxValue() {
42         if (isAce()) return 11;
43         else return value();
44     }
45
46     public boolean isAce() {
47         return faceValue == 1;
48     }
49
50     public boolean isFaceCard() {
51         return faceValue >= 11 && faceValue <= 13;
52     }
53 }

```

This is just one way of handling aces. We could, alternatively, create a class of type `Ace` that extends `BlackJackCard`.

An executable, fully automated version of blackjack is provided in the downloadable code attachment.

7.2 Call Center: Imagine you have a call center with three levels of employees: respondent, manager, and director. An incoming telephone call must be first allocated to a respondent who is free. If the respondent can't handle the call, he or she must escalate the call to a manager. If the manager is not free or not able to handle it, then the call should be escalated to a director. Design the classes and data structures for this problem. Implement a method `dispatchCall()` which assigns a call to the first available employee.

pg 127

SOLUTION

All three ranks of employees have different work to be done, so those specific functions are profile specific. We should keep these things within their respective class.

There are a few things which are common to them, like address, name, job title, and age. These things can be kept in one class and can be extended or inherited by others.

Finally, there should be one `CallHandler` class which would route the calls to the correct person.

Note that on any object-oriented design question, there are many ways to design the objects. Discuss the trade-offs of different solutions with your interviewer. You should usually design for long-term code flexibility and maintenance.

We'll go through each of the classes below in detail.

`CallHandler` represents the body of the program, and all calls are funneled first through it.

```
1  public class CallHandler {
2      /* 3 levels of employees: respondents, managers, directors. */
3      private final int LEVELS = 3;
4
5      /* Initialize 10 respondents, 4 managers, and 2 directors. */
6      private final int NUM_RESPONDENTS = 10;
7      private final int NUM_MANAGERS = 4;
8      private final int NUM_DIRECTORS = 2;
9
10     /* List of employees, by level.
11      * employeeLevels[0] = respondents
12      * employeeLevels[1] = managers
13      * employeeLevels[2] = directors
14      */
15     List<List<Employee>> employeeLevels;
16
17     /* queues for each call's rank */
18     List<List<Call>> callQueues;
19
20     public CallHandler() { ... }
21
22     /* Gets the first available employee who can handle this call.*/
23     public Employee getHandlerForCall(Call call) { ... }
24
25     /* Routes the call to an available employee, or saves in a queue if no employee
26      * is available. */
27     public void dispatchCall(Caller caller) {
28         Call call = new Call(caller);
29         dispatchCall(call);
30     }
31
32     /* Routes the call to an available employee, or saves in a queue if no employee
33      * is available. */
34     public void dispatchCall(Call call) {
35         /* Try to route the call to an employee with minimal rank. */
36         Employee emp = getHandlerForCall(call);
37         if (emp != null) {
38             emp.receiveCall(call);
39             call.setHandler(emp);
40         } else {
41             /* Place the call into corresponding call queue according to its rank. */
42             call.reply("Please wait for free employee to reply");
43             callQueues.get(call.getRank().getValue()).add(call);
44         }
45     }
```

```

46
47  / *An employee got free. Look for a waiting call that employee can serve. Return
48  * true if we assigned a call, false otherwise. */
49  public boolean assignCall(Employee emp) { ... }
50 }

```

Call represents a call from a user. A call has a minimum rank and is assigned to the first employee who can handle it.

```

1  public class Call {
2      / *Minimal rank of employee who can handle this call. */
3      private Rank rank;
4
5      / *Person who is calling. */
6      private Caller caller;
7
8      / *Employee who is handling call. */
9      private Employee handler;
10
11     public Call(Caller c) {
12         rank = Rank.Responder;
13         caller = c;
14     }
15
16     / *Set employee who is handling call. */
17     public void setHandler(Employee e) { handler = e; }
18
19     public void reply(String message) { ... }
20     public Rank getRank() { return rank; }
21     public void setRank(Rank r) { rank = r; }
22     public Rank incrementRank() { ... }
23     public void disconnect() { ... }
24 }

```

Employee is a super class for the **Director**, **Manager**, and **Respondent** classes. It is implemented as an abstract class since there should be no reason to instantiate an **Employee** type directly.

```

1  abstract class Employee {
2      private Call currentCall = null;
3      protected Rank rank;
4
5      public Employee(CallHandler handler) { ... }
6
7      / *Start the conversation */
8      public void receiveCall(Call call) { ... }
9
10     / *the issue is resolved, finish the call */
11     public void callCompleted() { ... }
12
13     / *The issue has not been resolved. Escalate the call, and assign a new call to
14     * the employee. */
15     public void escalateAndReassign() { ... }
16
17     / *Assign a new call to an employee, if the employee is free. */
18     public boolean assignNewCall() { ... }
19
20     / *Returns whether or not the employee is free. */
21     public boolean isFree() { return currentCall == null; }
22 }

```